arXiv:2305.11837v2 [cs.SE] 25 May 2023

COMPARING SOFTWARE DEVELOPERS WITH CHATGPT: AN EMPIRICAL INVESTIGATION

A PREPRINT

Nathalia Nascimento
David R. Cheriton School of Computer Science,
University of Waterloo, (UW)
Waterloo, N2L 3G1, Canada
nmoraesd@uwaterloo.ca

Paulo Alencar
David R. Cheriton School of Computer Science,
University of Waterloo, (UW)
Waterloo, N2L 3G1, Canada
palencar@uwaterloo.ca

Donald Cowan
David R. Cheriton School of Computer Science,
University of Waterloo, (UW)
Waterloo, N2L 3G1, Canada
dcowan@uwaterloo.ca

ABSTRACT

The advent of automation in particular Software Engineering (SE) tasks has transitioned from theory to reality. Numerous scholarly articles have documented the successful application of Artificial Intelligence to address issues in areas such as project management, modeling, testing, and development. A recent innovation is the introduction of ChatGPT, an ML-infused chatbot, touted as a resource proficient in generating programming codes and formulating software testing strategies for developers and testers respectively. Although there is speculation that AI-based computation can increase productivity and even substitute software engineers in software development, there is currently a lack of empirical evidence to verify this. Moreover, despite the primary focus on enhancing the accuracy of AI systems, non-functional requirements including energy efficiency, vulnerability, fairness (i.e., human bias), and safety frequently receive insufficient attention. This paper posits that a comprehensive comparison of software engineers and AI-based solutions, considering various evaluation criteria, is pivotal in fostering human-machine collaboration, enhancing the reliability of AI-based methods, and understanding task suitability for humans or AI. Furthermore, it facilitates the effective implementation of cooperative work structures and human-in-the-loop processes. This paper conducts an empirical investigation, contrasting the performance of software engineers and AI systems, like ChatGPT, across different evaluation metrics. The empirical study includes a case of assessing ChatGPT-generated code versus code produced by developers and uploaded in Leetcode.

Keywords Software Engineering · AI-based solutions · Performance Evaluation · ChatGPT · Machine Learning

1 Introduction

The popularity of AI-based tools such as ChatGPT (versions 3 and 4) [1, 2], a tool made available by OpenAI, has exploded. ChatGPT has set a record for the fastest-growing user, having reached 100 million users in January 2023 with 25 million daily visitors. As a result, the recent automation capabilities supported by ChatGPT resulted in increased interest in topics such as the potentially increasing the automation of software development tasks such as coding and testing [3, 4, 5] enabling programmers to make their tasks more efficient[2] and allowing them to be more productive, and assessing how human-machine teams that can be more effective in software development tasks [6, 7]. Indeed, tools such as ChatGPT have indeed led to impressive results both in terms of quantity and quality, and the produced outcomes (e.g., code) are in many cases comparable to what is produced by humans. For example, Golzadeh et al. performed an

investigation in large open-source projects on GitHub and observed that bots are among the most active contributors, without being labeled as bots [8].

However, while the research community has put in a great deal of effort to enhance the accuracy of AI-based approaches, they have often overlooked other non-functional requirements [9], such as energy efficiency [10], vulnerability [11], fairness (i.e. human bias) [12], and safety [5, 13]. According to Georgiou et al.[10], the use of deep learning frameworks has considerably increased energy consumption and CO2 emissions. For example, only "ChatGPT has been estimated to consume the equivalent of the electricity consumed by 175,000 people in Denmark per month" [5]. Pearce et al. [11] investigated the use of an AI-based code generator in security-relevant scenarios, and observed that 40% of the provided solutions were vulnerable. Sarro [5] brings attention to the presence of bias in various real-world systems relying on ML, such as advertisement and recruitment processes, and to safety problems, such as insecure code generation [13], as creating novel malware or inserting malware into the generated system, performance of dangerous operation such as file manipulation [14], or facilitating harmful acts, such as scamming, harassment, misinformation, and election interference. According to Sarro [5], not even a software engineer, regardless of their level of expertise, would be able to manually detect all possibilities for improving these non-functional characteristics.

In addition, while there is speculation that AI-based computation can increase productivity and even substitute software engineers in software development, there is currently a lack of empirical evidence to verify this [7]. In fact, there are few papers providing empirical studies to investigate the use of machine-learning techniques in Software Engineering and compare an ML-based solution against those provided by software engineers, particularly considering different non-functional properties. For example, Nascimento et al. [15] present an empirical study to compare software engineers to machine learning in the domain of the Internet of Things (IoT), addressing performance and reuse criteria, and conclude that "we cannot state that ML improves the performance of an application in comparison to solutions provided by IoT expert software engineers... Our experiment indicates that in some cases, software engineers outperform machine-learning algorithms, whereas in other cases, they do not."

Such an understanding is essential in realizing novel human-in-the-loop approaches in which AI procedures assist software developers in achieving tasks. Human-in-the-loop approaches, which take into account the strengths and weaknesses of humans and AI solutions, are fundamental not only to providing a basis for cooperative human-machine work or teams not only in software engineering but also in other application areas.

This paper presents an empirical study [16] to compare how software engineers and AI systems can be compared with respect to non-functional requirements such as performance and memory efficiency. The empirical study involves a case study assessing ChatGPT-generated code versus code produced by developers and uploaded in Leetcode, which consists of three steps: (i) we selected a contest from Leetcode that contains programming problems at different difficulty levels; (ii) we used these problems as prompts to ChatGPT to generate code; and (iii) we uploaded the ChatGPT code solution to Leetcode and compared them to the previous solutions based on performance and efficiency metrics.

This paper is organized as follows. Section 2 presents the related work. Section 3 presents the empirical study, describing research questions, hypotheses, and the objective of the study. Section 4 presents the experimental results and threats to validity. The paper ends in Section 5 with concluding remarks and suggestions for future work.

2 Related Work

Imai [7] claims that while there is speculation that AI-based computation can increase productivity and even substitute human pair programmers in software development, there is currently a lack of empirical evidence to verify this. In fact, there are few papers providing empirical studies to investigate the use of machine-learning techniques in Software Engineering and compare an ML-based solution against those provided by software engineers, particularly considering non-functional properties [15]. Imai [7] conducted an empirical study to compare the productivity and code quality between pair programming with GitHub Copilot and human pair programming. GitHub Copilot, a tool launched by OpenAI and GitHub, to provide code snippets and automatically fill in parts of code, gives users the choice to accept or reject its assistance depending on their knowledge. The experiment involved 21 participants, with each one receiving a project to code and a developer partner, either human or the GitHub Copilot. To evaluate, Imai [7] calculated productivity based on the number of lines of code added and code quality based on the number of lines of code removed after being added. The results showed that programming with Copilot helps generate more lines of code than human pair-programming in the same period of time, but the code quality was lower. Additionally, the author performed a preliminary evaluation of code confiability, as they reported that programmers tend to inspect the code generated by AI less than human pair-programmers.

Nguyen and Nadi [17] also conducted an empirical study using GitHub Copilot's generated code to assess the correctness and understandability of solutions for 33 Leetcode problems in four different programming languages. To evaluate the

correctness, the authors counted the number of test cases that passed for each problem, and to assess understandability, they employed SonarQube, an open-source platform for static code analysis, to calculate complexity and cognitive complexity metrics. The authors did not focus on performance and memory efficiency, so they did not provide execution time or memory use for each solution, nor did they compare the Copilot's and human-written solutions.

Li et al. presented AlphaCode [18], a code generation system. They trained their model using GitHub and CodeContests data. After using AlphaCode to solve competitive programming problems from the Codeforces platform, the authors state that "AlphaCode achieved on average a ranking of top 54.3% in competitions with more than 5,000 participants". They compared their solution against the developers' ones based on the contest metrics, which are a fraction of the time left in the contest and incorrect submission penalties. Lertbanjongngam et al. [19] extended AlphaCode evaluation. Using the same code provided by AlphaCode in [18] for the Codeforces competitive programming problems, they assessed human-like coding similarity, performance, and memory efficiency. Their results show that AlphaCode-generated codes are similar to human codes, having a uniqueness of less than 10% of code fragments. They also show that the code that was produced exhibits similar or inferior execution time and memory usage compared to the code written by humans. In contrast to Lertbanjongngam et al. [19], we utilized ChatGpt and randomly selected coding problems for our first experiment, where we compared the performance of code generators to that of human solutions. The tool was employed to generate code for each selected problem.

Nascimento et al. [15] conducted an empirical study to investigate the automation of a coding problem without a set of unit tests. Their experiment does not rely on match-based metrics such as generating a solution that passes a specific set of unit tests; instead, it is based on functional correctness, emphasizing that the solution should work efficiently. In addition, their experiment employs an unsupervised ML-based approach, where the ML approach offers a solution to the problem before it is presented to software engineers.

3 Experiment: ChatGPT vs Programmers - An Empirical Study Addressing Performance and Efficiency

Software engineers are constantly evaluated by their capability of problem-solving coding, which involves creating a program to solve a problem based on the problem specifications. Given a problem, they need to be able to understand what is being asked and write code to solve the problem. For years, this kind of task has been used to rank software engineers during job interviews and programming contests. Their evaluation usually relies on the number of right solutions, the performance of the solutions, memory efficiency, and development time.

For almost 76 years, researchers have been writing about the concept of "automatic programming" [20]. Recent advancements in language models, such as the Generative Pre-trained Transformer (GPT) series, have greatly advanced the field of automatic programming by enabling the generation of code, program synthesis, code completion, and bug detection. GPTs are deep neural networks that are pre-trained on vast amounts of natural language text and then fine-tuned for specific tasks such as question answering. Given a natural language description of a task, GPTs can generate a code that accomplishes the desired task, which is achieved by fine-tuning the model on a large corpus of code.

OpenAI's Codex is a language model built on the GPT architecture and integrated into ChatGPT. Chen et al. [13] introduced Codex and used a dataset of 163 coding problems to evaluate it, including introductory, interview, and competition problems. Their evaluation consisted of generating many solutions and checking if at least one passed the unit tests. Accordingly, after generating 100 samples per problem, their solution was able to solve 77.5% of the problems. They evaluated their approach based on efficacy (number of tests passed), not accessing performance, and memory efficiency. Imai [7], and Nguyen and Nadi [17] also performed empirical studies with a version of Codex. The non-functional requirements they evaluated are productivity, code quality, confiability, correctness, and understandability. We describe their findings in Section 2.

3.1 Objective

In this context, we decided to ask the following question: "How do software engineers compare with AI solutions with respect to performance and memory efficiency?" To explore this question, we selected coding problems as our target activity and compared a solution provided by an experienced contest programmer with a solution provided by ChatGPT. In short, Figure 1 depicts the theory [21] that we investigate in this paper. According to the theory, the variables that we intend to isolate and measure are the performance and memory efficiency achieved from three kinds of solutions: i) solutions provided by ChatGPT; ii) solutions provided by experience contest programmers; and iii) solutions provided by novice or non-contest programmers.

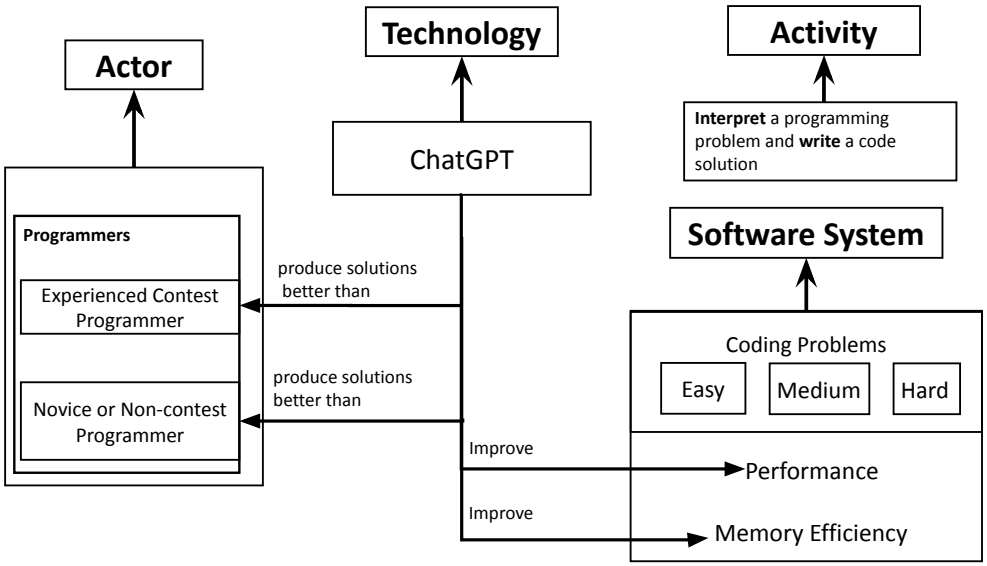

Figure 1: Theory [21]: ChatGPT outperforms programmers in problem-solving coding tasks with higher performance and memory efficiency.

To evaluate the relationship among these variables, we performed a controlled experiment using LeetCode, a well-established online platform for programming interview preparation and subsequently juxtaposed the solution presented by ChatGPT with those previously developed by software engineers. The next subsection describes the research questions (RQx) and the theory's propositions. To perform this experiment, we selected one of the most recent contests in LeetCode with novel coding problems [1]. Then, we used these problems as prompts to ChatGPT to generate code. Subsequently, the code produced by ChatGPT was uploaded to Leetcode, where it was evaluated against pre-existing solutions, utilizing performance and efficiency metrics. Given that Leetcode compares the submitted solution with all prior solutions to the same problem, we initiated a discussion on these metrics, predicated on programmer expertise. To this end, we selected a group of 42 programmers, differentiated by their rank in the contest, and categorized them according to the number of programming contests they have already attended in Leetcode.

3.2 Questions and Hypotheses

In the domain of problem-solving coding, how does the result from a ChatGPT solution differ from solutions provided by...

> **RQ1**. ...**experienced contest programmers** while solving easy, medium, or hard coding problems with respect to their **performance**?
>
> **RQ2**. ...**experienced contest programmers** while solving easy, medium, or hard coding problems with respect to their **memory efficiency**?
>
> **RQ3**. ...**novice contest programmers** while solving easy, medium, or hard coding problems with respect to their **performance**?
>
> **RQ4**. ...**novice contest programmers** while solving easy, medium, or hard coding problems with respect to their **memory efficiency**?

Each RQ is based on one or more hypotheses, which are described next. **H - RQ1**.

- H0. ChatGPT does not improve the performance of coding solutions compared to solutions provided by experienced contest programmers.
- HA. ChatGPT improves the performance of coding solutions compared to solutions provided by experienced contest programmers.

[1]The contest description is available at:
https://leetcode.com/contest/biweekly-contest-103/ (accessed on May 15 2023)

H - RQ2.

- H0. ChatGPT does not improve the memory efficiency of coding solutions compared to solutions provided by experienced contest programmers.
- HA. ChatGPT improves the memory efficiency of coding solutions compared to solutions provided by experienced contest programmers.

H - RQ3.

- H0. ChatGPT does not improve the performance of coding solutions compared to solutions provided by novice contest programmers.
- HA. ChatGPT improves the performance of coding solutions compared to solutions provided by novice contest programmers.

H - RQ4.

- H0. ChatGPT does not improve the memory efficiency of coding solutions compared to solutions provided by novice contest programmers.
- HA. ChatGPT improves the memory efficiency of coding solutions compared to solutions provided by novice contest programmers.

3.3 The Object of the Study: ChatGPT

By using ChatGPT, developers can generate code from natural language input, as shown in Figure 2. In addition to the code, the system can also provide an explanation and evaluation for the generated code, such as analyzing time and space complexity.

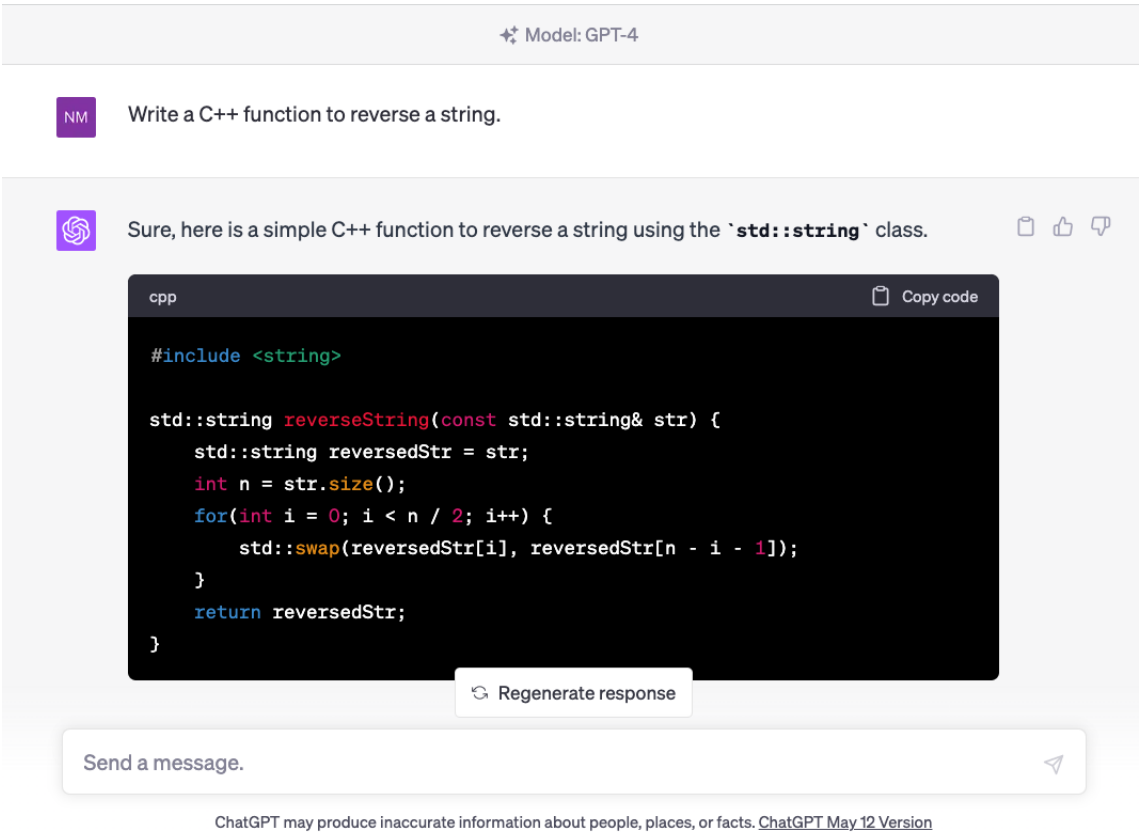

Figure 2: Example of using ChatGPT to solve a coding problem.

Although ChatGPT is capable of generating functional code for an expansive array of programming languages (including Java, Kotlin, Python, C++, JavaScript, TypeScript, PHP, Go-lang, Ruby, Swift, and more), we opted to focus on C++. This decision was taken by the fact that C++ is predominantly used by the most seasoned contest programmers. We used ChatGPT-4 to generate the code solutions.

3.4 Controlled Experiment

The initial phase of the experiment involved choosing a platform to select and implement the coding solutions. Our objective was to present both sets of solutions to the same application and assess the outcomes using an identical evaluation process. We opted for Leetcode, an online platform for preparing for coding interviews. It has more than 4 million users and provides more than 2000 coding problems and solutions, usually used by big tech companies to assess developers' skills and conduct interviews. Leetcode compares solutions based on performance (runtime execution) and memory usage.

We chose one of the most recent programming contests on Leetcode, which presented four unique programming challenges classified into one easy, two medium, and one hard problem. Out of the 17,137 participants, 12,493 managed to solve at least one problem. The number decreased to 10,733 for those who solved two problems, further reduced to 7,133 for three problems, and dwindled to a mere 700 for all four problems.

As illustrated in Figure 1, one of the factors we aim to isolate and quantify is the performance and memory efficiency of the solutions in relation to the programmer's skill level. To this end, we chose programmers based on their ranking in the contest, selecting those who used C++ and managed to solve at least three questions, ranging from the highest to the lowest rank. Utilizing LeetCode's parameters, we divided them into two categories: experienced contest programmers and novice or non-contest programmers. Ultimately, we juxtaposed the performance and memory efficiency of the solutions generated by ChatGPT against those provided by both groups of participants.

3.4.1 Participant Analysis

Figure 3 displays a distribution graph of the participants based on their ranking level on LeetCode, with lower levels indicating more experienced programmers. For instance, eight of the selected programmers fall within the top 1% as per LeetCode's ranking.

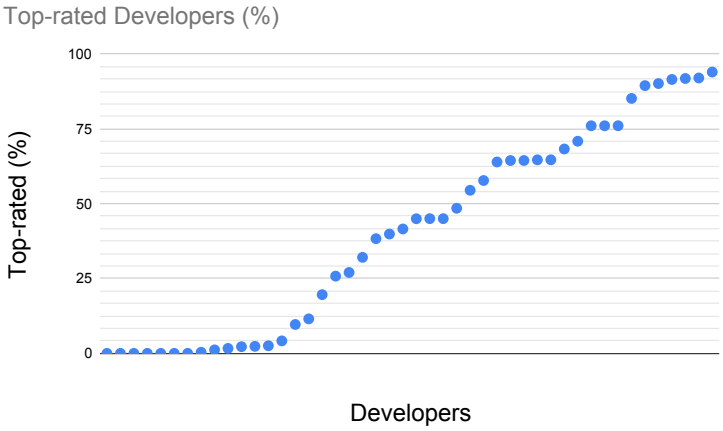

Figure 3: Distribution of participants based on LeetCode global ranking.

Programmers in the top 30% were grouped as Experienced Contest Programmers, skilled in problem-solving, algorithmic thinking, and coding languages from participating in numerous contests. They excel at quickly comprehending and solving problems under time pressure. The remaining participants were categorized as Novice or Non-contest Programmers, whose expertise lies more in software development tasks like design, coding, testing, and debugging. Despite their ability to solve complex problems, they may lack the speed and specialized algorithmic knowledge of contest programmers due to less exposure to competitive coding.

3.4.2 Experiment: Solutions provided by ChatGPT

We employed ChatGPT-4 to formulate a solution for each problem within the contest.[2]. The original descriptions of the problems were used as prompts, without any textual modifications to simplify the problem interpretation.

Table 3.4.2 encapsulates the list of problems along with the performance and memory efficiency of each solution. Upon running the solution on the LeetCode platform, it generates data on its runtime and memory efficiency. Furthermore, it provides a global ranking for each question, indicating the percentage of solutions outperformed by this solution in terms of performance and memory efficiency. These outcomes may fluctuate depending on LeetCode's processing demand. To ensure that the variation in time was inconsequential, we executed the same solution multiple times. We also evaluated the time and space complexity of the solutions. Interestingly, despite certain solutions demonstrating superior time complexity analysis, others showcased superior runtime performance and memory efficiency. This example is clearly demonstrated in the two solutions provided for the simplest problem. It is known that optimizing an application's performance depends not just on minimizing the algorithm's iterations, but also on selecting the most effective data structures.

Problem name	Difficulty	Laguage	Number of solutions that worked	If failed-number of test cases	Runtime (ms)	Memory (MB)	Performance (beats) (%)	Memory (beats) (%)	Time Complexity	Space Complexity	ID LeetCode submission
2656. Maximum Sum With Exactly K Elements	Easy	C++	1/1	-	47	70.6	70.22	68.69	O(n log n + k^2 log n)	O(n)	951082025
2656. Maximum Sum With Exactly K Elements	Easy	C++	2/3	-	62	71.7	21.75	12.19	O(n + k * log n)	O(n)	946957357
2657. Find the Prefix Common Array of Two Arrays	Medium	C++	1/1	-	53	80.9	78.35	97.47	O(n^3)	O(n)	951078174
2658. Maximum Number of Fish in a Grid	Medium	C++	1/1	-	84	88.4	69.14	88.14	O((mn)²)	O(mn)	951076414
2659. Make Array Empty	Hard	C++	0	505/514	-	-	-	-	O(n^2)	O(n)	951086982

Table 1: Result analysis of coding solutions provided by ChatGPT.

For some problems, ChatGPT-4 successfully generated a working solution on the first attempt. However, for others, multiple iterations were necessary to generate a solution that could pass all test cases. It is noteworthy that several software developers also had to submit multiple solutions before producing one that was accepted by Leetcode, passing all test cases.

The most challenging problem proved resistant, with none of the over 50 solutions generated (in various programming languages) able to pass all test cases. In this instance, the most effective solution generated by ChatGPT managed to pass 505 out of 514 test cases, falling short due to time limit exceedances for larger inputs. We posit that the subtlety of this question lies in the paradox between the textual description's suggestion of an algorithm to remove elements from an array and a hint indicating that the solution does not necessarily require the deletion or movement of elements, focusing instead on the array length. Essentially, it is anticipated that the developer's solution deviates from the direct interpretation of the problem description. Sobania et al. [22], in their evaluation of ChatGPT's code-generation capabilities against other similar solutions, noted that several problems which automated solutions could not solve were characterized by problem descriptions that were too ambiguous to be definitively solved, even by human programmers.

[2]The prompt and code solutions provided are available at:
Google Drive form (accessed on May 15 2023)

3.5 Experiment - Results

As detailed in Table 3.4.2, the solutions generated by ChatGPT-4 surpass 70.22%, 78.35%, and 69.14% of existing solutions for each respective problem. In terms of memory efficiency, the solutions given by ChatGPT-4 outpace 68.69%, 97.47%, and 88.14% of others, respectively. However, this comparison encompasses all previously provided solutions, thus lacking clarity on how these automated solutions perform relative to software engineers with varying expertise levels.

We ran the solutions supplied by both groups of developers to ascertain the specific runtime and memory-efficiency of their code. Table 2 showcases a selection of the results derived from the solutions that developers produced for the least complex problem. Tables 3, 4, and 5 address this by exclusively considering the solutions provided by our selected participants to the easy, first medium, and second medium problems, respectively [3].

Group	Developer Num	Global Ranking	Num of C++ problems solved	Contest Position	Top (%)	Runtime (ms)	Memory (MB)	Performance (beats) (%)	Memory (beats) (%)
A	5	21	2426	7	0.01	29	70.7	98.27	29.83
A	8	1335	55	18	0.37	36	70.7	93.71	68.73
A	9	2,915	1098	693	1.17	59	70.7	30.22	29.83
B	17	334,617	67	12477	85.17	70	70.6	9.8	68.73
B	18	351,789	140	7117	89.45	62	70.8	21.77	13.47
B	19	354,535	82	12487	90.14	52	70.7	52.49	29.83

Table 2: A selection of solution instances obtained from responses to the easy problem by the chosen participants (Groups A = Experienced; B = Novice).

Variable	N samples	Degrees of freedom (n-1)	t critical value (.99%)	Runtime - Best Value	Runtime - Mean	Runtime - Standard deviation	Runtime - tstatistic	Memory - Best Value	Memory - Mean	Memory - Std	Memory - tstatistic
Programmers	38	37	-2.43	29	54.63	16.60	-2.83	70.6	71.45	2.84	-1.86
Experienced contest programmers	20	19	-2.53	29	55	21.70	-1.64	70.6	72.035	3.83	-1.67
Novice contest programmers	18	17	-2.56	35	54.22	8.57	-3.57	70.6	70.81	0.55	-1.66
ChatGPT-4	1			47				70.6			

Table 3: Easy problem (2656. Maximum Sum With Exactly K Elements): data to perform test statistic - performance and memory efficiency.

Variable	N samples	Degrees of freedom (n-1)	t critical value (.99%)	Runtime - Best Value	Runtime - Mean	Runtime - Standard deviation	Runtime - tstatistic	Memory - Best Value	Memory - Mean	Memory - Std	Memory - tstatistic
Programmers	34	33	-2.44	48	118.5	123.03	-3.10	80.7	91.32	24.09	-2.52
Experienced contest programmers	19	18	-2.55	48	83.84	67.03	-2.00	80.7	87.66	15.86	-1.85
Novice contest programmers	15	14	-2.62	56	162.4	161.89	-2.61	81.1	95.95	31.68	-1.84
ChatGPT-4	1			53				80.9			

Table 4: Medium problem (2657. Find the Prefix Common Array of Two Arrays): data to perform test statistic - performance and memory efficiency.

Pertaining to the high-difficulty problem, it remained unsolved by both ChatGPT and novice contest programmers. Only the experienced contest programmers were successful in delivering an effective solution for this coding problem.

[3]The code provided by each participant and the submission links are available at:
Google Drive form (accessed on May 15, 2023)

Variable	N samples	Degrees of freedom (n-1)	t critical value (.99%)	Runtime - Best Value	Runtime - Mean	Runtime - Standard deviation	Runtime - tstatistic	Memory - Best Value	Memory - Mean	Memory - Std	Memory - tstatistic
Programmers	32	31	-2.45	56	154.71	95.34	-4.19	88.4	110.64	26.77	-4.69
Experienced contest programmers	19	18	-2.55	56	138.57	102.46	-2.32	88.4	105.32	26.57	-2.77
Novice contest programmers	13	12	-2.68	76	178.30	81.96	-4.148	88.4	118.40	26.12	-4.141
ChatGPT-4	1			84				88.4			

Table 5: Medium problem (2658. Maximum Number of Fish in a Grid): data to perform test statistic - performance and memory efficiency.

3.5.1 Hypothesis Testing

In this part, we investigate the hypotheses regarding the evaluation of solutions' performance and memory efficiency, as introduced in subsection 3.2. Consequently, we conducted statistical analyses, following the methods laid out by Peck and Devore [23], based on the metrics displayed in the three tables from Section 3.5.

We divided the experiment's outcomes into two categories: those from experienced contest programmers and those from novice contest programmers. Subsequently, we computed the mean and standard deviation of the results for each group. These statistical measures were then juxtaposed with the results obtained using ChatGPT. For instance, the first hypothesis (H - RQ1) posits that the solution generated by ChatGPT enhances application performance compared to solutions proposed by expert software engineers, leading to a reduction in runtime. The assertion, therefore, is that the mean runtime of expert software engineers' solutions surpasses that of the ChatGPT solution, which recorded 47ms for the easy problem, and 53ms and 84ms for the medium problems respectively.

Using a statistical significance level [23] of 0.01 (the chance of one in 100 of making an error), we computed the test statistic ($t - statistic$) for each one of the difficulty levels, as follows [23]:

$$t - statistic : t = \frac{(\overline{x} - hypothesizedvalue)}{(\frac{\sigma}{\sqrt{n}})} \tag{1}$$

Based on t-statistic theory, we can confidently reject our null hypothesis if the $t - statistic$ value falls below the negative $t - criticalvalue$ (threshold) [23]. This implies that if we had assessed the entire sample of selected participants without categorizing them into two groups, we could have found evidence suggesting that automated solutions, like ChatGPT, could potentially outperform programmers in general. As demonstrated in the three result tables, the $runtimet - statistic$ value is below the negative $t - criticalvalue$ value in all cases when considering the entire group of programmers. The same holds true for the $memoryt - statistic$ in the medium-level problems.

3.6 Discussion

In our empirical investigation, where we evaluated the performance and memory efficiency of solutions offered by programmers of varying experience levels, we validated three alternative hypotheses while three were rejected:

Accepted:

1. For easy and medium-level problems, ChatGPT enhances the performance of coding solutions in comparison to those provided by novice contest programmers.
2. For medium-level problems, ChatGPT enhances memory efficiency in coding solutions in comparison to those provided by both experienced and novice contest programmers.

Rejected:

1. ChatGPT enhances the performance of coding solutions for easy, medium, or hard-level problems when compared to those provided by experienced contest programmers.
2. For easy and hard-level coding problems, ChatGPT demonstrates superior memory efficiency when compared to the solutions offered by either novice or experienced contest programmers.

These findings suggest that automated solutions, such as ChatGPT, may outperform software engineers in certain software engineering tasks. Specifically, ChatGPT demonstrated superior performance over novice contest programmers in solving easy and medium-level problems and also exhibited better memory efficiency in one of the medium-level problems.

However, we found no evidence to assert that ChatGPT surpasses the performance of solutions provided by experienced contest programmers. This insight is notable given the significant research interest in the automation of software development tasks.

In summary, our study suggests a nuanced relationship between the performance of software engineers and AI-based solutions: in certain scenarios, software engineers excel, while in others, AI proves superior. This underscores the importance of understanding the unique strengths of both human and automated approaches, facilitating more effective collaborative work and task allocation processes [24]. The findings also emphasize the need for AI systems with adaptable degrees of automation, in line with the perspective offered by Melo et al. [25]. Within a software engineering context, this suggests adjusting the level of AI automation based on both the experience of the developer and the quality requirements of the task at hand.

3.7 Threats to Validity

The exact training and testing data employed by ChatGPT remains undisclosed, meaning we cannot ascertain if our queries' precise solutions already exist within the data. Consequently, the specific coding challenges used to train their tool remain unidentified. Even though we opted for a recent LeetCode contest with novel problems, we cannot ensure that we did not test problems similar to or the same as those used in training their algorithm. As such, the tool may not be creating a fresh solution but could be retrieving a previously stored solution for a specific problem.

Moreover, there's the potential that software engineers have previously leveraged automated systems to submit questions to LeetCode, without giving due credit. As such, we cannot assert that the comparisons made were exclusively with solutions provided by developers. For instance, Golzadeh et al. have presented evidence suggesting that bots regularly feature among the most active contributors on GitHub, despite GitHub not recognizing their contributions [8]. To mitigate this issue, we implemented participant selection.

4 Conclusion and Future Work

Several researchers have proposed the use of AI systems to automate software engineering tasks. However, most of these approaches do not direct efforts toward asking whether AI-based procedures have higher success rates than current standard and manual practices. A relevant question in this potential line of investigation is: "Could a software engineer solve a specific development task better than an automated system?". Indeed, it is fundamental to evaluate which tasks are better performed by engineers or AI procedures so that they can work together more effectively and also provide more insight into novel human-in-the-loop AI approaches to support SE tasks.

Though there is conjecture that AI-based computation could enhance productivity and potentially replace software engineers in software development, current empirical evidence supporting this claim is scant. Indeed, a limited number of papers offer empirical investigations into the application of machine-learning techniques in Software Engineering. This paper introduces an empirical study examining the utilization of automated strategies like ChatGPT to automate a task in Software Engineering, specifically, solving coding problems. Moreover, as inherent in experimental studies, even with careful design and execution, certain factors could pose threats to experimental validity. One such potential threat includes the precise training and testing data employed by ChatGPT.

Our empirical study uncovered that automated systems like ChatGPT can, in certain instances, surpass the performance of novice software engineers in specific tasks. This superiority was particularly evident in the solving of easy and medium-level problems, where ChatGPT's performance consistently exceeded that of novice contest programmers. Moreover, the AI-based solution demonstrated improved memory efficiency for a medium-level problem. In contrast, we found no substantial evidence to suggest that ChatGPT could outdo experienced contest programmers in terms of solution performance. In essence, our study reveals a dynamic interplay between human and AI performance in software engineering tasks, highlighting the need for different task allocation processes. This encourages a collaborative approach, fine-tuning AI assistance based on developer expertise and task quality requirements.

This empirical investigation ought to explore the potential for automation in software engineering tasks extending beyond the realm of problem-solving in coding. Future work to extend the proposed experiment includes: (i) conducting further empirical studies to assess other SE tasks, such as design, maintenance, testing, and project management; (ii) experimenting with other AI approaches, such as unsupervised machine-learning algorithms; and (iii) using different

criteria to evaluate task execution, addressing different qualitative or quantitative methodologies. Possible tasks that could be investigated (refer to (i)) include testing tasks (e.g. comparing the number, type and difficulty level of faults that were identified by developers), designing tasks (i.e. accessing system usability), maintenance tasks (i.e. accessing continuous performance), and project management tasks (i.e. evaluating the level of satisfaction of developers in the task allocation process).

Acknowledgment

This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC), and the Centre for Community Mapping (COMAP).

References

[1] OpenAI. Gpt-4 technical report. *https://doi.org/10.48550/arXiv.2303.08774*, 2023.

[2] Tyna Eloundou, Sam Manning, Pamela Mishkin, and Daniel Rock. Gpts are gpts: An early look at the labor market impact potential of large language models, 2023.

[3] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C Schmidt. Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. *https://doi.org/10.48550/arXiv.2303.07839*, 2023.

[4] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. An analysis of the automatic bug fixing performance of chatgpt. *https://doi.org/10.48550/ARXIV.2301.08653*, 2023.

[5] Federica Sarro. Automated optimisation of modern software system properties. In *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*, pages 3–4, 2023.

[6] Glaucia Melo, Luis Fernando Lins, Paulo Alencar, and Donald Cowan. Supporting contextual conversational agent-based software development. *International Conference on Software Engineering*, 2023.

[7] Saki Imai. Is github copilot a substitute for human pair-programming? an empirical study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pages 319–321, 2022.

[8] Mehdi Golzadeh, Tom Mens, Alexandre Decan, Eleni Constantinou, and Natarajan Chidambaram. Recognizing bot activity in collaborative software development. *IEEE Software*, 39(5):56–61, 2022.

[9] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623, 2021.

[10] Stefanos Georgiou, Maria Kechagia, Tushar Sharma, Federica Sarro, and Ying Zou. Green ai: Do deep learning frameworks have different costs? In *Proceedings of the 44th International Conference on Software Engineering*, pages 1082–1094, 2022.

[11] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768. IEEE, 2022.

[12] Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan. A survey on bias and fairness in machine learning. *ACM Comput. Surv.*, 54(6), jul 2021.

[13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[14] Jeevana Priya Inala, Chenglong Wang, Mei Yang, Andres Codas, Mark Encarnación, Shuvendu Lahiri, Madanlal Musuvathi, and Jianfeng Gao. Fault-aware neural code rankers. *Advances in Neural Information Processing Systems*, 35:13419–13432, 2022.

[15] Nathalia Nascimento, Paulo Alencar, Carlos Lucena, and Donald Cowan. Toward human-in-the-loop collaboration between software engineers and machine learning algorithms. In *2018 IEEE International Conference on Big Data (Big Data)*, pages 3534–3540. IEEE, 2018.

[16] Steve Easterbrook, Janice Singer, Margaret-Anne Storey, and Daniela Damian. Selecting empirical methods for software engineering research. *Guide to advanced empirical software engineering*, pages 285–311, 2008.

[17] Nhan Nguyen and Sarah Nadi. An empirical evaluation of github copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories*, pages 1–5, 2022.

[18] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

[19] Sila Lertbanjongngam, Bodin Chinthanet, Takashi Ishio, Raula Gaikovina Kula, Pattara Leelaprute, Bundit Manaskasemsak, Arnon Rungsawang, and Kenichi Matsumoto. An empirical evaluation of competitive programming ai: A case study of alphacode. In *2022 IEEE 16th International Workshop on Software Clones (IWSC)*, pages 10–15. IEEE, 2022.

[20] F Brooks and HJ Kugler. *No silver bullet*. April, 1987.

[21] Dag IK Sjøberg, Tore Dybå, Bente CD Anda, and Jo E Hannay. Building theories in software engineering. *Guide to advanced empirical software engineering*, pages 312–336, 2008.

[22] Dominik Sobania, Martin Briesch, and Franz Rothlauf. Choose your programming copilot: a comparison of the program synthesis performance of github copilot and genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 1019–1027, 2022.

[23] Roxy Peck and Jay Devore. *Statistics: The Exploration & Analysis of Data*. Nelson Education, 2011.

[24] Nathalia Nascimento, Paulo Alencar, and Donald Cowan. An approach to support human-in-the-loop big data software development projects. In *2021 IEEE International Conference on Big Data (Big Data)*, pages 2319–2326. IEEE, 2021.

[25] Glaucia Melo, Nathalia Nascimento, Paulo Alencar, and Donald Cowan. Understanding levels of automation in human-machine collaboration. In *2022 IEEE International Conference on Big Data (Big Data)*, pages 3952–3958. IEEE, 2022.